

Ok if electron app is done then give me a pdf about all at features about in the pdf how to make these features comptabile with Android, like the proxy model etc and evrything clean white animated auth page like white and black see attachments things that are good and don't need changes, the auth page looks clean neat, but then comes the main page which appear very broken things arranged poorly and the list container components should be in side bar which user can toggle to show like other ai apps, not at bottom, + the cheap emojis used fix it, + one more thing now tell that agent to seperate electron and website and keep only apk, build same keep things like auth page etc same do tweaks improve fixes, + terminal is fake native terminal termux node pty, posix, has its pkg, and own directory binaries compiled Android SDK 28 to bypass the SDK and has comptability like termux has its binary manager pkg which used to install packages all real no fake, powerful like termux full terminal commands, has its own directory on users device file and prefix like termux, do research then make this pdf, + the page main page chat page the chat box but bigger big clean improved the background bit animated or whatever, if u need figma ui libraries for bit tweaks or components then u can use clear modern design, here the figma components url
https://www.figma.com/community/file/1645103147299035222/claude-redesign-concept-approachable-intelligence?q_id=ecac7ac3-1c7f-43b3-b937-7cc44e832141 this is give some modern claude app designs which u gonna use, and here another
https://www.figma.com/community/file/1353986262501321287/google-gemini-ai-app-version?q_id=2e43131a-fb2c-4aef-843a-bac48065611a this is gemini app components library which u gonna use use specially from them phrasing designs etc and modernity premiumness clear and sharp impression eye soothing and every satisfying and clear to see and use. No black colour background background must be consistent white as it is, look how premium the auth page looks how designed so other things would look must good also arrange things in main page i. Order, 3 line at top which open side bar, expandable containers like tools > with arrow grey text or any text same keep consistent font, and below that expandable containers, this will arrange up things, and each things has its own dedicated page open, not all mess in one main page, and also the playground page is suck completely build it from scratch in apk, ur not buliding i mean ur Makin pdf for our next session ai to build it inside that repo, i select, so make the researched detailed prompt pdf here more keys or do some research and use your own knowledge and things, don't compromise, add things if u like to add in pdf extra features key things or etc etcWe are re-architecting the entire background execution engine to run a coordinated, multi-process AI Agent Workflow using JetBrains' Koog framework and the Model Context Protocol (MCP) standards. You are strictly forbidden from writing hardcoded thread timers, mock execution logs, or static class layouts. All agent capabilities must be represented as explicit, schema-backed tool registrations that process real background operations.

Implement the core system components step-by-step:

AUTOMATED OPEN-SOURCE TOOLING & INSTALLATION:

Write the automated NDK compilation scripts and Gradle configuration blocks to download, compile, and install the following open-source frameworks directly into our project container:

OPENCV FOR ANDROID: Native C++ library compiled for the 'arm64-v8a' target, providing local frame manipulation.

TERMUX-API: Native shell hardware bridges, providing terminal control over device features (battery, vibration, tts, flashlight).

TERMUX-PUPPETEER-MCP: Set up an automated bash script to initialize a rootless 'proot-distro' Alpine container, install Chromium and Node.js inside it, and establish the local Puppeteer execution bridge on port 9222.

SQLITE-VEC: Cross-compile the 'sqlite-vec' C-extension and link it directly to our database layer to support local, offline vector embedding similarity queries.

THE 15-SKILL MCP SCHEMA LAYER (THE CONTRACTS):

You must write the explicit tool definition schemas using strongly-typed Kotlin data classes and JSON models. Do not mock these contracts. Define the exact input/output parameters for each of these 15 Agent Skills:

agent_read_file

agent_write_file

agent_grep_search

agent_execute_command (routes directly to the low-level JNI forkpty() master/slave bridge)

puppeteer_screenshot (runs headless Chromium inside the Alpine proot-distro)

puppeteer_navigate

puppeteer_click

sqlite_vec_query (queries vec0 tables inside sqlite-vec)

android_dump_ui_tree (queries active AccessibilityNodeInfo structures)

android_inject_gesture (dispatches programmatically via dispatchGesture())

android_launch_intent

termux_api_notify

android_ocr_inspect (crops frame buffer and runs Tesseract4Android)

process_state_diagnose (invokes IActivityManager via reflection)

device_policy_manager (binds to Dhizuku binder channel)

THE MULTI-PROCESS KOOG SUB-AGENT WORKFLOWS:

Implement the background service binders to run these eight sub-agent classes concurrently:

GAMING-BASED AGENT: Coordinates continuous MediaProjection frame capture, OpenCV template matching, and Accessibility gesture injection.

CODING AGENT: Performs low-level C++/Kotlin file editing and on-device NDK toolchain compilation.

DEBUGGING AGENT: Monitors active compiler streams, tracks crashes, and captures device logcat diagnostics.

RESEARCHER AGENT: Accesses an isolated PRoot-Distro container to drive headless Puppeteer/Chromium, bypassing web anti-bot blocks silently in the background.

PLANNING AGENT: Parses the screen's structural JSON schema and dynamically outlines step-by-step task instructions.

REFINING AGENT: Reviews output data structures, validates compiler compliance, and optimizes payloads.

TASK EXECUTION AGENT: Manages our native pseudoterminal process loop, invoking commands and installing packages.

DAILY WORKER AGENT: Scrapes system app content (such as reading Gmail text nodes) via Accessibility Services to automate daily tasks.

BINDER INTEGRATION & STATE CHECKPOINTS:

Implement the AIDL service binders ('IInputService.aidl', 'ISystemControlService.aidl') to bridge communication between our application, Shizuku user services, and the administrative Dhizuku client.

Configure Koog's directed graph state tracker to automatically checkpoint active workflows at each node. If an agent process is killed, restore state parameters from the last saved node checkpoint rather than replaying from step 1. The 15 Real AI Agent Skills (MCP Tools) Explained

1. agent_read_file (Workspace File Reader)

The Skill Schema: Receives a dynamic target path parameter and optional line limits.

The Native Action: Bypasses visual UI restrictions to read the raw text contents of files inside the application's workspace directory.[1] It returns the exact data to the agent's context window.[1]

2. agent_write_file (Workspace File Patch/Writer)

The Skill Schema: Accepts a target file path and a raw code block string.

The Native Action: Automatically writes or surgically patches files (such as build.gradle.kts or MainActivity.kt) natively on the device filesystem without manual editing.[1]

3. agent_grep_search (Regex Code Scanner)

The Skill Schema: Receives a regex string pattern and directory filters.

The Native Action: Recursively scans the workspace code files to locate specific variable definitions, class setups, or layout parameters, matching IDE search performance.[2]

4. agent_execute_command (Shell Execution Helper)

The Skill Schema: Takes a raw system or build command string.

The Native Action: Spawns an actual background process inside the master/slave Pseudoterminal (PTY) bridge.[3, 4] It runs builds, manages packages, or initiates scripts, returning the raw standard output/error directly back to the agent.

5. puppeteer_screenshot (Virtual Screen Capture)

The Skill Schema: Receives a webpage URL and options like viewport size.

The Native Action: Automatically fires up an Alpine or Debian container inside our rootless proot-distro sandbox, launches a headless Chromium instance, captures a JPEG screenshot of the rendered DOM, and converts it to a compact Base64 image payload.

6. puppeteer_navigate (Headless DOM Crawler)

The Skill Schema: Accepts a target URL string.

The Native Action: Navigates to any dynamic, JavaScript-heavy website using the headless Chromium bridge. It evaluates the client-side JavaScript, handles active cookie assets, and returns the fully compiled HTML DOM tree.

7. puppeteer_click (Interactive Web Operator)

The Skill Schema: Takes a web selector or element coordinate parameter.

The Native Action: Interacts dynamically with web elements (such as clicking buttons, submitting logins, or selecting dropdown items) inside the headless browser container.

8. sqlite_vec_query (Semantic Memory Retrieval)

The Skill Schema: Accepts a high-dimensional query embedding vector (obtained from local model generation).

The Native Action: Queries the on-device SQLite database containing the native sqlite-vec extension. It runs a K-nearest neighbor similarity query across specialized vec0 virtual memory tables to fetch episodic task history or long-term preferences instantly.

9. android_dump_ui_tree (Accessibility Node Inspector)

The Skill Schema: Requires no parameters; returns the active screen tree.

The Native Action: Intercepts the system UI context via our active background AccessibilityService. It dumps the active window node hierarchy, returning specific resource ID paths (such as reading message threads in Gmail or address bars in Chrome) without relying on fragile OCR.

10. android_inject_gesture (Implicit Touch Controller)

The Skill Schema: Accepts touch coordinate mappings (X, Y) and swipe path durations.

The Native Action: Synthesizes precise hardware-level touches, taps, or continuous swipes programmatically. This bypasses Android obscuration overlays, interacting with apps or games exactly like a real physical finger.

11. android_launch_intent (Dynamic App Controller)

The Skill Schema: Accepts an application package name or system action string.

The Native Action: Invokes high-level platform intents to open target applications (like Chrome, Roblox, or settings screens) directly from the background, managing the system task stack seamlessly.

12. termux_api_notify (Haptic & System Communicator)

The Skill Schema: Receives title, message, and haptic feedback duration parameters.

The Native Action: Interacts with the native termux-api package to trigger haptic vibrations, launch system-level notifications, or query battery health directly from background scripts.

13. android_ocr_inspect (Visual Area Grounding)

The Skill Schema: Receives bounding box parameters (left, top, right, bottom).

The Native Action: Grabs the active screen frame buffer from our double-buffered MediaProjection pipeline, crops the image to the specified coordinate boundaries, and executes Tesseract4Android locally to extract clean text data.

14. process_state_diagnose (Hidden OS Diagnostic)

The Skill Schema: Accepts a target application package identifier.

The Native Action: Interfaces with the hidden system class IActivityManager using Java reflection via a Shizuku binder proxy.[5, 6] It extracts the exact exit reasons (such as Low Memory, ANR, or user force-stop) for background tasks.[6]

15. device_policy_manager (Dhizuku Admin Operator)

The Skill Schema: Accepts an APK path and restriction parameter flags.

The Native Action: Integrates directly with the DevicePolicyManager binder transaction channel via the Dhizuku daemon.[3, 7] It silently installs updates, configures background execution limits, and locks settings without requiring root.[3, 7]Based on a rigorous analysis of your mobile console screenshots, active system logs, and the deep systems architecture we have mapped out, here is the complete, core-to-core technical breakdown of the ****15 Native Operational Skills****.

These skills are designed specifically to operate within your custom package namespace (com.pecker.mobilestudio.vpkzqd) using a zero-simulation, event-driven state model. Each explanation details exactly how the background system handles the execution, how it interacts with the Android kernel, and how it resolves the visual and logical errors seen in your app's screens.

The 15 Operational Skills Explained (Core-to-Core)

1. Direct JNI PTY Spawning Skill

* **The System Mechanism:** This skill handles spawning a true, interactive shell (like bash or dash) instead of a basic Java byte pipe. It uses a native C++ JNI library to invoke the low-level Linux system call `forkpty()` or `openpty()`.^[4]

* **Why It's Essential:** Without this, running interactive tools like nano or ssh throws a "not a typewriter" (`ioctl TTY`) error and instantly crashes. By allocating a master/slave pseudoterminal (PTY) in the Android kernel, it attaches the slave device to the terminal processes and channels raw character streams directly to your user interface viewport.

2. LD_PRELOAD Path Redirection Skill

* **The System Mechanism:** Since official Termux packages are precompiled with a hardcoded path prefix `/data/data/com.termux/files/usr/` [1, 2], this skill injects a compiled dynamic shared library (`libtermux_exec.so`) into the runtime environment.^[5]

* **Why It's Essential:** It hooks critical system-level functions (like `execve`, `open`, `openat`, and `stat`) using standard dynamic linking intercepts (`RTLD_NEXT`).^[6] When any command attempts to access a hardcoded Termux file path, our shim intercepts the call mid-flight and redirects it to our app's private folder:

This bypasses directory mismatch linker errors natively without altering a single byte of the compiled packages.

3. Recursive File Unpacker Skill (Fixes Image 1 EISDIR Crash)

* **The System Mechanism:** This is the exact fix for the crash shown in your start-up log: `open failed: EISDIR (Is a directory)`.

* **Why It's Essential:** The original file unpacking routine was attempting to open directory entities as flat file streams. This skill refactors the decompression loop (handling `.tar.gz` or `.zip` archives) by implementing strict, recursive verification:

```
```kotlin

if (entry.isDirectory) {

 File(targetDirectory, entry.name).mkdirs()

} else {
```

```

entry.inputStream.use { input ->

 File(targetDirectory, entry.name).outputStream().use { output ->

 input.copyTo(output)

 }

}

}

}

...

```

Once the directory tree is programmatically created, the skill loops through the unpacked bin directory and recursively executes `File.setExecutable(true)` (equivalent to running `chmod +x` at the system layer) so the binary tools are runnable by the shell.

#### #### 4. On-Device Vision Inference (ExecuTorch) Skill

**\* \*\*The System Mechanism:\*\*** This skill manages on-device machine learning object-detection networks. It bypasses cloud-dependent AI processing by cross-compiling Meta's ExecuTorch with ARM KleidiAI and XNNPack backends.[7]

**\* \*\*Why It's Essential:\*\*** It loads a highly optimized, quantized target model (like YOLOv8n) into memory. By leveraging the Snapdragon NPU/cDSP, it maps on-screen bounding boxes, character coordinates, and player positions locally with ultra-low latency, running entirely offline.

#### #### 5. OpenCV Transparent Masking & Template Matching Skill

**\* \*\*The System Mechanism:\*\*** This is a low-level C++ OpenCV native framework. It runs highly optimized image processing functions directly inside native memory buffers rather than converting them to heavy Java Bitmap objects.

**\* \*\*Why It's Essential:\*\*** It uses the `cv::matchTemplate()` function with custom transparent alpha channel masks. This allows the system to identify overlapping, semi-transparent game HUD controls, attack buttons, or dynamic interface indicators reliably, regardless of complex or shifting screen backgrounds.

#### #### 6. Continuous MediaProjection Capture Skill

\* \*\*The System Mechanism:\*\* This skill manages the real-time, continuous video pipeline. It registers a persistent, background Android ForegroundService configured with the strict system flag:

android:foregroundServiceType="mediaProjection" [8, 9]

\* \*\*Why It's Essential:\*\* When the user grants permission, it creates a VirtualDisplay linked to an ImageReader using the GPU-native PixelFormat.RGBA\_8888 color space.[8] By directly draining frame buffers at 60\text{ FPS} into local memory arrays, it bypasses slow, laggy CPU rendering loops to feed our OpenCV and ExecuTorch engines with low latency.

#### #### 7. Accessibility XML Tree Scraper Skill

\* \*\*The System Mechanism:\*\* When enabled, this background AccessibilityService receives a continuous, real-time structured XML representation of everything displayed on the screen.[10]

\* \*\*Why It's Essential:\*\* Rather than relying on fragile OCR processing, the agent traverses the active tree node-by-node, reading exact, named resource identifiers directly (e.g., retrieving com.google.android.gm:id/mail\_message\_body inside Gmail to read message bodies, or com.android.chrome:id/url\_bar in Chrome to scrape URLs).[10] This allows silent and secure on-screen context scraping across any active system application.

#### #### 8. Implicit Touch Gesture Injector Skill

\* \*\*The System Mechanism:\*\* Operating inside our authorized accessibility context, this skill programmatically synthesizes physical hardware-level inputs.[11]

\* \*\*Why It's Essential:\*\* It utilizes the dispatchGesture() API to send clicks, swipes, and multi-touch interactions directly to the screen's input coordinator. Because these touches are injected directly at the system's input manager layer [10], they completely bypass application-level clickjacking security filters (like filterTouchesWhenObscured), allowing the agent to play games, click prompts, and control screens exactly like a human user.[10, 12]

#### #### 9. PRoot Virtualization Engine Skill

\* \*\*The System Mechanism:\*\* This skill integrates the official, custom-patched Termux proot utility from the package tree into your app sandbox.

\* \*\*Why It's Essential:\*\* By using the Linux kernel's ptrace system call [3], it intercepts child process system calls and translates directory references.[3] This virtualizes a complete Linux root directory (/) within your app's sandbox namespace, allowing you to run precompiled Linux packages and full userlands (like Ubuntu or Alpine) seamlessly on unrooted devices.

#### #### 10. Headless Puppeteer Scraper Skill

\* \*\*The System Mechanism:\*\* This skill is the ultimate background web crawling system. It spawns an Alpine or Debian container within our rootless proot-distro environment.

\* \*\*Why It's Essential:\*\* It runs a headless Node.js/Puppeteer script that controls an ARM-compiled desktop Chromium browser locally on the device. The browser runs silently in the background, executing scripts, resolving dynamic AJAX elements, and solving anti-bot measures to scrape pages without displaying a visible window.

#### #### 11. SQLite Vector Memory (sqlite-vec) Skill

\* \*\*The System Mechanism:\*\* To give your autonomous sub-agents persistent memories without sending data to the cloud, this skill cross-compiles the lightweight C-extension sqlite-vec directly into your application's database.

\* \*\*Why It's Essential:\*\* It allows your database engine to store high-dimensional text embeddings in specialized vec0 virtual tables and run K-nearest neighbor (KNN) similarity queries locally on-device. This provides your agents with rapid episodic and semantic memory lookup capabilities at native speed.

#### #### 12. Local DB SQLite Engine Skill

\* \*\*The System Mechanism:\*\* This is a fully native, local SQLite database layer packaged directModels, <https://openrouter.ai/blog/insights/llm-gateway/> 3. How to Add Authentication to an Electron App - OpenReplay Blog, <https://blog.openreplay.com/add-authentication-electron-app/> 4. Deep Links | Electron, <https://electronjs.org/docs/latest/tutorial/launch-app-from-url-in-another-app> 5. Deep Linking With Our React Native App | by nikhil Khadilkar | Razorpay Engineering, <https://engineering.razorpay.com/deep-linking-with-our-react-native-app-9cbee7fdcbd7> 6. Cursor - TrueFoundry Docs, <https://www.truefoundry.com/docs/ai-gateway/cursor> 7. "Override OpenAI Base URL" breaks requests when pointing to OpenRouter - Bug Reports, <https://forum.cursor.com/t/override-openai-base-url-breaks-requests-when-pointing-to-openrouter/142520> 8. Using the AppAuth PKCE to Authenticate to your Electron Application, <https://developers.onelogin.com/api-authorization/using-the-appauth-pkce-to-authenticate-to-your-electron-application> 9. Custom Protocols and Deeplinking in Electron apps - projects, <https://blog.bloomca.me/2025/07/20/electron-apps-custom-protocols.html> 10. Cursor – agentgateway | Agent Connectivity Solved, <https://agentgateway.dev/docs/kubernetes/1.0.x/integrations/llm-clients/cursor/> 11. One Key, One Bill? A Practical Guide to LLM Gateways - WebsiteInit, <https://websiteinit.com/blog/llm-gateways-one-key-one-bill/> 12. Achieving Sub-Millisecond Proxy Overhead - LiteLLM, <https://docs.litellm.ai/blog/sub-millisecond-proxy-overhead> 13. January — Real-time OpenAI Response Streaming with Node.js, <https://january.sh/posts/real-time-openai-response-streaming-node-js> 14. GitHub - diegosouzapw/OmniRoute: Never stop coding. Free MIT AI gateway: one endpoint, 290+

providers (90+ free), 500+ models — Kimi, Claude, GPT, OpenAI, Gemini, GLM, DeepSeek, MiniMax. Works with Claude Code, Codex, Cursor, OpenCode, Cline & Copilot. Quota-aware auto-fallback, RTK+Caveman compression saves 15-95% tokens, MCP/A2A, Desktop/PWA. Built by 500+ contributors, diegosouzapw/OmniRoute 15. LiteLLM and LiteLLM Enterprise - ALM Toolbox, <https://www.almttoolbox.com/litellm> 16. Custom API Endpoints for Cursor, Claude Code, Cline & More (2026) - OfoxAI, <https://ofox.ai/blog/cursor-claude-code-cline-custom-api-setup-2026/> 17. Cursor Integration - LiteLLM, [https://docs.litellm.ai/docs/tutorials/cursor\\_integration](https://docs.litellm.ai/docs/tutorials/cursor_integration) 18. Cursor Enterprise Network Proxy Configuration: Complete Guide from HTTP\_PROXY to Certificate Import - Easton Dev, <https://eastondev.com/blog/en/posts/ai/20260529-cursor-proxy-config/> 19. Building deep-links in Electron application | BigBinary Blog, <https://www.bigbinary.com/blog/deep-link-electron-app> 20. Does electron's registerHttpProtocol work in development? - Stack Overflow, <https://stackoverflow.com/questions/68838012/does-electrons-registerhttpprotocol-work-in-development> 21. ElectronJS Protocol - Open your app from a URL - YouTube, [https://www.youtube.com/watch?v=l0kt4rRjF\\_A](https://www.youtube.com/watch?v=l0kt4rRjF_A) Architecting a Multi-Platform AI Gateway Client: System Design, Protocol Normalization, and Native Integration Framework

Deploying an enterprise-grade AI gateway across desktop and mobile environments requires unified API translation, secure protocol handling, and resilient streaming architecture. Integrating OmniRoute-inspired routing patterns into an application framework—targetable to both Electron desktop shells and Android APK runtimes—presents complex system design challenges. These include managing multi-provider request/response translation, enforcing OAuth 2.0 Proof Key for Code Exchange (PKCE) authentication flows over deep links, implementing zero-latency Server-Sent Events (SSE) proxy layers, and addressing non-standard client behavior in IDEs such as Cursor.

This research report provides a comprehensive blueprint for constructing a cross-platform gateway client and embedded routing engine. It examines multi-provider API translation, native deep-link callback mechanisms, real-time streaming pipelines, relational state schemas, and platform compatibility targets.

## 1. Cross-Platform Gateway Architecture and Core System Topology

A multi-platform AI client must bridge native host capabilities with a localized, OpenAI-compatible AI routing core. The system architecture operates across three functional layers: the Native Runtime Container, the Embedded Gateway Engine, and the External Model Provider Ecosystem. The native runtime container abstracts host-level interactions, leveraging Electron for desktop environments and Capacitor or native Android wrappers for mobile APK deployments. Within this container, the embedded gateway engine functions as a localized background daemon exposing a standardized RESTful and streaming surface at `/v1/*`, mirroring the OpenAI specification to ensure seamless client compatibility.

The architectural topology dictates that on desktop systems, Electron hosts the gateway as an embedded background process. Inter-process communication (IPC) bridges communicate between the native user interface and the core, while local loopback IP bindings (<http://127.0.0.1:PORT/v1>) allow external local development tools and IDEs to query the gateway directly. On Android APK runtimes, the gateway operates within a foreground service wrapper or compiled native binary layer, handling requests issued by the mobile user interface or local network peers while managing battery and memory constraints.

The primary responsibility of the embedded gateway is absorbing provider credential management, rate limits, dialect variations, and model capability differences. Rather than requiring applications to maintain separate integration logic for hundreds of discrete model providers, the application routes all interactions through the internal proxy engine. This engine dynamically selects upstream credentials, translates JSON structures, manages exponential backoff retries, and normalizes stream payloads before delivering output to the client application.

## 2. Multi-Provider Translation Engine and Streaming Proxy Mechanics

The core duty of an OmniRoute-style gateway is translating unified request structures into provider-specific schemas, executing calls over HTTP/2 or HTTP/1.1, and streaming responses back without introducing proxy overhead. Request processing begins at the entry route (`src/sse/handlers/chat.ts`), which routes payloads into the core orchestration engine (`open-sse/handlers/chatCore.ts`). The translation registry (`open-sse/translator/index.ts`) evaluates format detection rules (`open-sse/services/provider.ts`) to map incoming payloads to the targeted executor adapter (`open-sse/executors/*`).

Upstream providers diverge significantly in their expectations for chat payloads, system role definitions, tool call declarations, and structured output formats. The engine employs a Strategy Pattern across executor modules to parse and convert incoming requests. OpenAI-native APIs recognize system, developer, user, assistant, and tool roles. Upstream providers like Anthropic or legacy Google Gemini endpoints strictly enforce alternating user and model/assistant roles, while rejecting developer or duplicate system entries. The translation engine (`open-sse/services/roleNormalizer.ts`) map-flattens consecutive same-role messages, transforms developer roles into system instructions, or injects leading system prompts into the top-level request body as required by the destination API.

Structured output conversions translate standard OpenAI `json_schema` definitions into target-native formats, such as Google Gemini's `responseSchema` or Anthropic's structured tool invocation blocks. For reasoning models that output internal thought chains enclosed in `<think>...</think>` tags, a stream parser (`open-sse/utils/thinkTagParser.ts`) intercepts raw chunks. The engine separates reasoning blocks from standard content blocks, emitting them either as extended telemetry metadata or stripping them to preserve compatibility with downstream clients expecting clean string tokens.

Real-time response delivery relies on Server-Sent Events (SSE). When proxying streams via Next.js App Routes or Node.js HTTP servers, memory spikes and socket stalls are prevented by streaming chunks through a TransformStream pipeline. The streaming pipeline processes incoming raw chunks, buffers incomplete JSON lines, converts provider-specific SSE formats (such as Anthropic's content\_block\_delta or Gemini's array chunks) into standard data: {"choices": [{"delta": {"content": "..."}]} events, and flushes them down the wire. Stream sanitization (open-sse/handlers/responseSanitizer.ts) filters mid-flight data to strip broken control characters or malformed tool arguments before they reach the consumer SDK. Backpressure enforcement relies on native Node.js or Web ReadableStream controllers. If the client socket slows down (detected via write() returning false or high highWaterMark backpressure in web streams), the gateway pauses upstream stream reading until the client drains its buffer, preventing unbuffered memory consumption in the gateway process.

To maximize uptime and cost efficiency, the routing layer relies on a multi-tiered resilience framework. Requests are routed dynamically based on explicit combo fallbacks or OmniRoute's 12-factor Auto-Combo scoring engine (open-sse/services/autoCombo/). The Auto-Combo engine evaluates candidate provider accounts using live metrics: provider operational health, remaining quota headroom, reset window proximity, historic latency, request success rates, cost per token, rate-limit proximity, and prompt-cache alignment. When an upstream endpoint returns a retrievable error (such as HTTP 429, 502, or 503), the gateway's circuit breaker (open-sse/services/circuitBreaker.ts) marks the account co-provider deployment as temporarily degraded. An exponential backoff cooldown window (src/sse/services/cooldownAwareRetry.ts) is applied, and the request is automatically retried through the next account in the priority chain. Mutex locking on account selection prevents thundering herd conditions across concurrent requests.

### 3. IDE App Integration Protocols and Quirks Normalization

Connecting local IDE tools (such as Cursor, Cline, Windsurf, Claude Code, and Zed) to the embedded local gateway provides developers with unified model access and centralized cost management. However, variations in client payload structures require provider normalization at the proxy boundary.

Target Client

Target Endpoint Route

Client Config Location / Variable

Primary Payload Format / Dialect

Known Quirks & Remediation

Cursor IDE

`http://127.0.0.1:PORT/v1` or `/cursor`

Settings > Models > Override OpenAI Base URL

Responses API / Anthropic native structure over `/chat/completions`

Sends non-standard fields (input, previous\_response\_id, flat tool definitions). Requires `/cursor` normalization bridge and disabling HTTP/2.

Cline (VS Code)

`http://127.0.0.1:PORT/v1`

Extension Settings > Provider: OpenAI Compatible

Standard OpenAI Chat Completions JSON

Strict role checking; rejects unknown parameters. Strips unsupported top-level fields in gateway translator.

Windsurf

`http://127.0.0.1:PORT/v1`

Settings > AI > Custom Model Provider

Standard OpenAI Chat Completions JSON

Requires exact model string matching. Utilize gateway model aliasing map.

Claude Code CLI

`h[span_64](start_span)[span_64](end_span)http://127.0.0.1:PORT/v1` or native

`ANTHROPIC_BASE_URL`, `ANTHROPIC_API_KEY` env vars

Native Anthropic `/v1/messages` format

Proxy must handle direct Anthropic API structures without forcing OpenAI schema transformations.

Zed Editor

`http://127.0.0.1:PORT/v1`

`settings.json > language_models.openai.api_url`

Standard OpenAI Chat Completions JSON

Minimal tolerance for chunk malformation in SSE. Ensure sanitization middleware is active.

Cursor handles base URL overrides differently depending on the configured backend dialect. When configured with an overridden base URL, Cursor often emits requests to `/chat/completions` containing structural anomalies. When configured for OpenAI models, Cursor may transmit an input array rather than a standard `messages` array, inject `previous_response_id` parameters, include store parameters, or pass non-standard tool parameters. When configured for Anthropic models, Cursor wraps native Anthropic payload structures inside a `/chat/completions` call. Furthermore, certain enterprise proxy configurations or internal networking layers stall during long-lived SSE streaming over HTTP/2, causing Agent mode requests to hang indefinitely.

To resolve these issues, the embedded gateway includes a dedicated `/cursor` endpoint adapter. This module intercepts incoming requests, detects the presence of input or Anthropic fields, maps input objects back into standard messages arrays, strips unsupported top-level parameters, and normalizes output stream events. Additionally, developers should configure `cursor.general.disableHttp2: true` in Cursor's `settings.json` to force stable HTTP/1.1 chunked transport.

The gateway also acts as a Model Context Protocol (MCP) server bridge, allowing tools like Cursor or Cline to execute external integrations dynamically. By maintaining an `mcp.json` registry, the proxy exposes available capabilities—such as web search, image generation, filesystem access, and terminal execution—to IDE clients via standard JSON-RPC 2.0 transport over stdio or SSE.

#### 4. Universal Authentication, OAuth 2.0 PKCE, and Deep Link Routing

Supporting diverse AI providers requires handling both static API keys and dynamic OAuth 2.0 access tokens across providers such as Google Vertex, GitHub Copilot, Codex, or custom enterprise SSO instances. Secure OAuth token acquisition in desktop and mobile applications requires implementing Proof Key for Code Exchange (PKCE, RFC 7636) alongside native deep link handlers. Static client secrets cannot be embedded securely within client-side application

binaries. The PKCE flow secures public client authorization through a dynamically generated cryptographic keypair.

The PKCE authentication sequence begins when the application creates a cryptographically random, high-entropy string  $V$  in [43, 128] characters using unreserved URI characters. The application then calculates the SHA-256 hash of the verifier and Base64URL-encodes the result:

$$\text{Challenge} = \text{Base64URL}(\text{SHA256}(V))$$

Next, the client launches the user's default system browser pointing to the provider's authorization endpoint, passing the client identifier, redirect URI, code challenge, and challenge method. After successful user authentication, the identity provider redirects to the application's registered custom URI scheme (such as `omniroute://oauth/callback?code=AUTH_CODE`). The native application intercepts this deep link, extracts the authorization code, and issues a direct HTTP POST request to the provider's token endpoint, supplying the unhashed code verifier  $V$ . The authorization server hashes the verifier, matches it against the stored challenge, and returns access and refresh tokens to the local client.

Deep link handling varies by target platform and requires explicit registration to route callbacks back to the running application instance. Electron applications register custom protocol schemes during startup using `app.setAsDefaultProtocolClient('omniroute')`. Operating systems process incoming URL schemes differently. On macOS, the operating system emits the `open-url` event on the existing application process. On Windows and Linux, the operating system attempts to launch a secondary application instance, passing the full callback URL in `process.argv`.

To maintain a single application instance on Windows and Linux, the main process calls `app.requestSingleInstanceLock()`. If the lock acquisition fails, the new instance immediately terminates. The running primary instance intercepts the second-instance event, parses `process.argv` to extract the authentication URL, and restores focus to the main window. On Android, mobile deep linking uses Custom URL Schemes or Android App Links. App Links require hosting a Digital Asset Links JSON file at `https://domain.com/.well-known/assetlinks.json` to verify domain ownership and prevent interception by third-party applications. When the authorization server redirects to the callback URI, the Android operating system resumes the primary activity, enabling the application to extract the authorization payload, execute the token exchange, and clear sensitive parameters from memory.

## 5. Persistence, Relational Schemas, and Cloud State Synchronization

The embedded gateway relies on a local relational state storage engine. Local persistence is handled by SQLite (`storage.sqlite`), supplemented by log text files (`log.txt`) and call payload archives (`call_logs/`).

Table Name

Key Schema Columns

Functional Description

Encryption Policy

providers

id, slug, name, base\_url, auth\_type,  
en[span\_37](start\_span)[span\_37](end\_span)[span\_41](start\_span)[span\_41](end\_span)abled

Tracks provider metadata, capabilities, and execution handlers.

Public metadata. Plaintext.

connections

id, provider\_id, account\_name, api\_key, access\_token, refresh\_token, expires\_at, quota\_limit,  
current\_spend

Stores raw credentials, OAuth token states, and usage metrics per account.

High Sensitivity. AES-256-GCM encrypted.

virtual\_keys

id, key\_[span\_68](start\_span)[span\_68](end\_span)hash, name, allowed\_models, budget\_limit,  
rate\_limit\_rpm, is\_active

Manages localized application/IDE authentication keys and budget caps.

Medium Sensitivity. Hash comparison (key\_hash).

model\_combos

id, combo\_name, fallback\_sequence\_json, strategy, composite\_tiers

Defines execution chains, fallback order, and load balancing policies.

Standard metadata. Plaintext.

usage\_telemetry

id, virtual\_key\_id, connection\_id, model\_used, prompt\_tokens, completion\_tokens, total\_cost, latency\_ms, timestamp

Records granular request metrics for local analytics and budget tracking.

Audit log dataset. Plaintext.

auto\_combo\_decisions

id, request\_id, winning\_connection\_id, candidate\_scores\_json, decision\_reason, timestamp

Records scoring calculations made by the 12-factor Auto-Combo engine.

System audit trail. Plaintext.

webhooks

id, event\_type, target\_url, secret, retry\_count, created\_at

Manages outbound webhook dispatching for provider status events.

Medium Sensitivity. HMAC secret encrypted.

All stored API keys, OAuth access tokens, and refresh tokens are encrypted before writing to SQLite. On desktop builds, encryption keys are derived using operating system keychains, such as Apple Keychain, Windows Credential Manager, or Linux Secret Service API accessed via Electron's safeStorage API. On Android runtimes, keys are derived using the Android Keystore system. The derived master key encrypts sensitive connection strings using AES-256-GCM with a unique 12-byte initialization vector (IV) generated per record.

To synchronize model configurations, provider priority tiers, and usage limits across multi-device deployments, the embedded engine integrates an optional Cloud Sync agent communicating with `NEXT_PUBLIC_CLOUD_URL`. State synchronization uses delta updates calculated as:

$$\Delta S = S_{\text{local}} \setminus S_{\text{remote}}$$

Local state changes generate signed JSON patch records. When network connectivity is established, these patches sync to the cloud control plane. Conflict resolution prioritizes local user modifications over cloud configurations, ensuring operational autonomy if cloud endpoints become unavailable.

## 6. Planning, Platform Compatibility, and Feature Specifications

Building an OmniRoute-compatible application requires accounting for performance and capability differences across target platforms.

| Architecture Feature | Customizability Level | Upstream Protocol Impact | Desktop Execution Context | Mobile (APK) Execution Context | | :--- | :--- | :--- | :--- | tool that creates a personalized audio summary of news articles I provide, so I can listen on my computer it can read full PDFs story tell any time etc, full files etc. it can read articles etc. add a beautiful web browsing feature where you intergate open source browser or any powerful workarounds where user can search websites and things in our app itself. If possible. No mockup or bug, and shows daily news article in clean ui container, showing latest articles. Not just predefined thing, get latest news> true, fetch news latestly create secret pipeline flow, any type of, and user can read and tell ai to summarise and tell or read for user etc etc and more configuration and advanced features from your side as wellOk in playground few more things and tools to add Build a tool that generates complete email marketing campaigns, including subject lines, body copy, and visuals, from a prompt.and more configuration and features for that according to you,. More advanced and more concept you add from your side hereOk cursor, we are going to make a massive project so he precise clear and hardworking, no laziness, + be sharp, ok build an app called oceanstudio. Our course insight is this application built with most powerful coding languages available, like python electron, and etc etc. fully lost powerful supported native language. This is coding agent which works on hardware level to code on hardware, so choose the right language which gives more advancements capabilities and features, however once this application built with electron wether exe then during building keep in mind that afterwards we might make this application copy in apk, and website format too, with supported features like that, but that would be later in the end. For now let's focus on core things, first the ui, see connect the repo ocean.studio and setup your cloud agents first, once evrything done then let's go ui, in your mcp connection i have implemented firebase, using that go in firebase create the accurate project in firebase and connect authentication, build our authentication page with premium quality and design. White with soft and very little reddish or maybe yellowish tone, soft full white. Or white and black whatever, feels like real modern site, no terminal like fake uis mock up simulations, the auth flow bar should be dynamically animated and transitionful, and must work, use can sign up, sign in manually, can directly sign in with Google. Also set-up forget password etc all must work already,once granted user land in the modern clean ui landing page, with lot of text footer and detail about our features. And u can use background looped videos or images, or any 3d components from different services of you need, for animated design eye soothing, you do not add useless cards like ui in the middle of the screen anywhere uselessly, you download stock videos or footages for quality and use them as neededfor premium site look, then a launch workspace button gives. Which first ask user few configuration, like the ui of the templates of how it looks, 3-4, and then what language like English or etc any with good transition, then review driven agent, auto agent or bypassed agent. These features must work, auto agent think before proceeding and be cautious on critical things that could harm. While review driven agent do evrything on users approval, and bypassed do all indepently without any review or anything, i

want this to work clean, just like you cursor look how good u made I want my site to be better than cursor and have cursor functionalits more and antigravity and vscode functionalities and more, with proper research no simulation but actual workflow just like big companies.first of all then the main page open, user get a side bar on left clean thing, with not outline gradients but shadow grey bit of shadow on the outline scroll driven scroll bar must work in sidebar so features can be expanded and seen. Then in the middle user get editor or preview screen, the editor is where codes present, user can edit it rewrite, delete or ,make any changes to them, by choosing the code file. Then it would show in that screen middle, cleanly phrased, and beside that at top in small px components user get option like code base, preview, the codebase shows all the codebase files etc in the environment, cleanly real, and preview shows hardware driven preview of sites, apps, websites, exes, and anything, nd preview also has functionality to works on port, if user activate or wether our agent actiavte any port, via using its shell, the user does not need to open the browser, it would directly see the port in our preview and use it whatever it is, smoothly. Full screen, give back environment to exit preview, in preview and codebase contains all code files folder, dynamically all with modern ui functionalities, like don't add emojis, design evrything user can expand code folders to see files or folders inside then and expand other folder inside then to see more folder or file like full functionality, ok this is done. Then in right side, add a right side bar, this right sidebar plate, is where our chatbox, agent chatbox, evrything lives, in it create downbelow in bottom a curved with no hard corners chatbox, with grey shadow outline or maybe RGB rim lighting on the outline, in loop, cleanly, send button in chatbox cleanly, and + button to open some things I'll talk later, then when user send any prompt or massage then the pause button shows to terminate the session, and user can type again, i mean normal things with all ai including u have these functionalities,now the + icon, the + icon has open a list of things,like mcp connectors, webhooks, plugins, upload photo, upload file, this feature must work how other ai works, however we talk about plugin webhooks and mcp connectors later but upload file and upload photo, thees must work how other ais works including you, same logic, do proper researches ok? Now first integrate a terminal in our app and agent internallr/slave terminal bridge.

puppeteer\_screenshot & puppeteer\_navigate & puppeteer\_click: Runs headless Chromium inside the Alpine container to navigate, render, and analyze website souomplete, production-grade systems foundation that is fully capable of running continuous unrooted automation!, making these all features work without keep main page clean. Just chatbox and at top sidebar button or etc whatever, and in chatbox, model selection, send button voice button and the + button and here bit proxy thing see if it's important or whatever Architecting a Multi-Platform AI Gateway Client: System Design, Protocol Normalization, and Native Integration Framework

Deploying an enterprise-grade AI gateway across desktop and mobile environments requires unified API translation, secure protocol handling, and resilient streaming architecture. Integrating OmniRoute-inspired routing patterns into an application framework—targetable to both Electron desktop shells and Android APK runtimes—presents complex system design challenges. These include managing multi-provider request/response translation, enforcing OAuth 2.0 Proof Key for Code Exchange (PKCE) authentication flows over deep links,

implementing zero-latency Server-Sent Events (SSE) proxy layers, and addressing non-standard client behavior in IDEs such as Cursor.

This research report provides a comprehensive blueprint for constructing a cross-platform gateway client and embedded routing engine. It examines multi-provider API translation, native deep-link callback mechanisms, real-time streaming pipelines, relational state schemas, and platform compatibility targets.

## 1. Cross-Platform Gateway Architecture and Core System Topology

A multi-platform AI client must bridge native host capabilities with a localized, OpenAI-compatible AI routing core. The system architecture operates across three functional layers: the Native Runtime Container, the Embedded Gateway Engine, and the External Model Provider Ecosystem. The native runtime container abstracts host-level interactions, leveraging Electron for desktop environments and Capacitor or native Android wrappers for mobile APK deployments. Within this container, the embedded gateway engine functions as a localized background daemon exposing a standardized RESTful and streaming surface at `/v1/*`, mirroring the OpenAI specification to ensure seamless client compatibility.

The architectural topology dictates that on desktop systems, Electron hosts the gateway as an embedded background process. Inter-process communication (IPC) bridges communicate between the native user interface and the core, while local loopback IP bindings (`http://127.0.0.1:PORT/v1`) allow external local development tools and IDEs to query the gateway directly. On Android APK runtimes, the gateway operates within a foreground service wrapper or compiled native binary layer, handling requests issued by the mobile user interface or local network peers while managing battery and memory constraints.

The primary responsibility of the embedded gateway is absorbing provider credential management, rate limits, dialect variations, and model capability differences. Rather than requiring applications to maintain separate integration logic for hundreds of discrete model providers, the application routes all interactions through the internal proxy engine. This engine dynamically selects upstream credentials, translates JSON structures, manages exponential backoff retries, and normalizes stream payloads before delivering output to the client application.

## 2. Multi-Provider Translation Engine and Streaming Proxy Mechanics

The core duty of an OmniRoute-style gateway is translating unified request structures into provider-specific schemas, executing calls over HTTP/2 or HTTP/1.1, and streaming responses back without introducing proxy overhead. Request processing begins at the entry route (`src/sse/handlers/chat.ts`), which routes payloads into the core orchestration engine (`open-sse/handlers/chatCore.ts`). The translation registry (`open-sse/translator/index.ts`) evaluates format detection rules (`open-sse/services/provider.ts`) to map incoming payloads to the targeted executor adapter (`open-sse/executors/*`).

Upstream providers diverge significantly in their expectations for chat payloads, system role definitions, tool call declarations, and structured output formats. The engine employs a Strategy Pattern across executor modules to parse and convert incoming requests. OpenAI-native APIs recognize system, developer, user, assistant, and tool roles. Upstream providers like Anthropic or legacy Google Gemini endpoints strictly enforce alternating user and model/assistant roles, while rejecting developer or duplicate system entries. The translation engine (`open-sse/services/roleNormalizer.ts`) map-flattens consecutive same-role messages,

transforms developer roles into system instructions, or injects leading system prompts into the top-level request body as required by the destination API.

Structured output conversions translate standard OpenAI json\_schema definitions into target-native formats, such as Google Gemini's responseSchema or Anthropic's structured tool invocation blocks. For reasoning models that output internal thought chains enclosed in `<think>...</think>` tags, a stream parser (`open-sse/utils/thinkTagParser.ts`) intercepts raw chunks. The engine separates reasoning blocks from standard content blocks, emitting them either as extended telemetry metadata or stripping them to preserve compatibility with downstream clients expecting clean string tokens.

Real-time response delivery relies on Server-Sent Events (SSE). When proxying streams via Next.js App Routes or Node.js HTTP servers, memory spikes and socket stalls are prevented by streaming chunks through a TransformStream pipeline. The streaming pipeline processes incoming raw chunks, buffers incomplete JSON lines, converts provider-specific SSE formats (such as Anthropic's content\_block\_delta or Gemini's array chunks) into standard data: `{"choices": [{"delta": {"content": "..."}]}` events, and flushes them down the wire. Stream sanitization

(`open-sse/handlers/responseSanitizer.ts`) filters mid-flight data to strip broken control characters or malformed tool arguments before they reach the consumer SDK. Backpressure enforcement relies on native Node.js or Web ReadableStream controllers. If the client socket slows down (detected via `write()` returning false or high `highWaterMark` backpressure in web streams), the gateway pauses upstream stream reading until the client drains its buffer, preventing unbuffered memory consumption in the gateway process.

To maximize uptime and cost efficiency, the routing layer relies on a multi-tiered resilience framework. Requests are routed dynamically based on explicit combo fallbacks or OmniRoute's 12-factor Auto-Combo scoring engine (`open-sse/services/autoCombo/`). The Auto-Combo engine evaluates candidate provider accounts using live metrics: provider operational health, remaining quota headroom, reset window proximity, historic latency, request success rates, cost per token, rate-limit proximity, and prompt-cache alignment. When an upstream endpoint returns a retrievable error (such as HTTP 429, 502, or 503), the gateway's circuit breaker (`open-sse/services/circuitBreaker.ts`) marks the account co-provider deployment as temporarily degraded. An exponential backoff cooldown window (`src/sse/services/cooldownAwareRetry.ts`) is applied, and the request is automatically retried through the next account in the priority chain. Mutex locking on account selection prevents thundering herd conditions across concurrent requests.

### 3. IDE App Integration Protocols and Quirks Normalization

Connecting local IDE tools (such as Cursor, Cline, Windsurf, Claude Code, and Zed) to the embedded local gateway provides developers with unified model access and centralized cost management. However, variations in client payload structures require provider normalization at the proxy boundary.

Target Client

Target Endpoint Route

Client Config Location / Variable

Primary Payload Format / Dialect

## Known Quirks & Remediation

### Cursor IDE

`http://127.0.0.1:PORT/v1` or `/cursor`

Settings > Models > Override OpenAI Base URL

Responses API / Anthropic native structure over `/chat/completions`

Sends non-standard fields (input, `previous_response_id`, flat tool definitions). Requires `/cursor` normalization bridge and disabling HTTP/2.

### Cline (VS Code)

`http://127.0.0.1:PORT/v1`

Extension Settings > Provider: OpenAI Compatible

Standard OpenAI Chat Completions JSON

Strict role checking; rejects unknown parameters. Strips unsupported top-level fields in gateway translator.

### Windsurf

`http://127.0.0.1:PORT/v1`

Settings > AI > Custom Model Provider

Standard OpenAI Chat Completions JSON

Requires exact model string matching. Utilize gateway model aliasing map.

### Claude Code CLI

`h[span_64](start_span)[span_64](end_span)tp://127.0.0.1:PORT/v1` or native

`ANTHROPIC_BASE_URL`, `ANTHROPIC_API_KEY` env vars

Native Anthropic `/v1/messages` format

Proxy must handle direct Anthropic API structures without forcing OpenAI schema transformations.

### Zed Editor

`http://127.0.0.1:PORT/v1`

`settings.json` > `language_models.openai.api_url`

Standard OpenAI Chat Completions JSON

Minimal tolerance for chunk malformation in SSE. Ensure sanitization middleware is active.

Cursor handles base URL overrides differently depending on the configured backend dialect.

When configured with an overridden base URL, Cursor often emits requests to

`/chat/completions` containing structural anomalies. When configured for OpenAI models, Cursor may transmit an input array rather than a standard

`me[span_67](start_span)[span_67](end_span)ssages` array, inject `previous_response_id`

parameters, include store parameters, or pass non-standard tool parameters. When configured

for Anthropic models, Cursor wraps native Anthropic payload structures inside a

`/chat/completions` call. Furthermore, certain enterprise proxy configurations or internal

networking layers stall during long-lived SSE streaming over HTTP/2, causing Agent mode

requests to hang indefinitely.

To resolve these issues, the embedded gateway includes a dedicated `/cursor` endpoint adapter.

This module intercepts incoming requests, detects the presence of input or Anthropic fields,

maps input objects back into standard messages arrays, strips unsupported top-level

parameters, and normalizes output stream events. Additionally, developers should configure

`cursor.general.disableHttp2: true` in Cursor's `settings.json` to force stable HTTP/1.1 chunked transport.

The gateway also acts as a Model Context Protocol (MCP) server bridge, allowing tools like Cursor or Cline to execute external integrations dynamically. By maintaining an `mcp.json` registry, the proxy exposes available capabilities—such as web search, image generation, filesystem access, and terminal execution—to IDE clients via standard JSON-RPC 2.0 transport over stdio or SSE.

#### 4. Universal Authentication, OAuth 2.0 PKCE, and Deep Link Routing

Supporting diverse AI providers requires handling both static API keys and dynamic OAuth 2.0 access tokens across providers such as Google Vertex, GitHub Copilot, Codex, or custom enterprise SSO instances. Secure OAuth token acquisition in desktop and mobile applications requires implementing Proof Key for Code Exchange (PKCE, RFC 7636) alongside native deep link handlers. Static client secrets cannot be embedded securely within client-side application binaries. The PKCE flow secures public client authorization through a dynamically generated cryptographic keypair.

The PKCE authentication sequence begins when the application creates a cryptographically random, high-entropy string  $V$  in [43, 128] characters using unreserved URI characters. The application then calculates the SHA-256 hash of the verifier and Base64URL-encodes the result:

$$\text{Challenge} = \text{Base64URL}(\text{SHA256}(V))$$

Next, the client launches the user's default system browser pointing to the provider's authorization endpoint, passing the client identifier, redirect URI, code challenge, and challenge method. After successful user authentication, the identity provider redirects to the application's registered custom URI scheme (such as `omniroute://oauth/callback?code=AUTH_CODE`). The native application intercepts this deep link, extracts the authorization code, and issues a direct HTTP POST request to the provider's token endpoint, supplying the unhashed code verifier  $V$ . The authorization server hashes the verifier, matches it against the stored challenge, and returns access and refresh tokens to the local client.

Deep link handling varies by target platform and requires explicit registration to route callbacks back to the running application instance. Electron applications register custom protocol schemes during startup using `app.setAsDefaultProtocolClient('omniroute')`. Operating systems process incoming URL schemes differently. On macOS, the operating system emits the `open-url` event on the existing application process. On Windows and Linux, the operating system attempts to launch a secondary application instance, passing the full callback URL in `process.argv`.

To maintain a single application instance on Windows and Linux, the main process calls `app.requestSingleInstanceLock()`. If the lock acquisition fails, the new instance immediately terminates. The running primary instance intercepts the second-instance event, parses `process.argv` to extract the authentication URL, and restores focus to the main window. On Android, mobile deep linking uses Custom URL Schemes or Android App Links. App Links require hosting a Digital Asset Links JSON file at `https://domain.com/.well-known/assetlinks.json` to verify domain ownership and prevent interception by third-party applications. When the authorization server redirects to the callback URI, the Android operating system re